Name (Last, First): Answer Key

These problems are about `37k_spinlocks` and `38k_semaphores`: your spinlock, the semaphore built on it, and the kernel heap. There are 3 problems on 6 pages. Answer each part in the space below it, and show your work.

1. **Spinlocks (37k).** Here is the core of the 37k `SpinLock`:

```
pub struct SpinLock<T> { locked: AtomicBool, data: UnsafeCell<T> }

pub fn lock(&self) -> SpinLockGuard<'_, T> {          // spin until we win
    while self.locked
        .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)
        .is_err()
    { core::hint::spin_loop(); }
    SpinLockGuard { lock: self }
}
pub fn try_lock(&self) -> Option<SpinLockGuard<'_, T>> {  // ONE attempt, no spinning
    if self.locked.compare_exchange(                  // in one atomic step:
        false, true,                                  //   if locked is false, set it to true
        Ordering::Acquire, Ordering::Relaxed,
    ).is_ok() {                                       // Ok: we set it, so the lock is ours
        Some(SpinLockGuard { lock: self })            //   hand back a guard
    } else {
        None                                          // Err: it was already true (held)
    }
}
fn unlock(&self) { self.locked.store(false, Ordering::Release); }

impl<T> Drop for SpinLockGuard<'_, T> {     // dropping a guard releases the lock
    fn drop(&mut self) { self.lock.unlock(); }
}

static COUNTER: SpinLock<u64> = SpinLock::new(0);
```

(a) A student writes `try_lock` without `compare_exchange`:

```
pub fn try_lock(&self) -> Option<SpinLockGuard<'_, T>> {
    if !self.locked.load(Ordering::Acquire) {          // A: is it free?
        self.locked.store(true, Ordering::Relaxed);    // B: claim it
        Some(SpinLockGuard { lock: self })
    } else { None }
}
```

Two CPUs call `COUNTER.try_lock()` at the same time. Give an order of steps `A` and `B` on the two CPUs in which *both* get a guard. What can go wrong once both CPUs hold a guard? Why does `compare_exchange` fix it?

CPU 1: A (sees false) → CPU 2: A (sees false) → CPU 1: B → CPU 2: B. Both saw "free", so both return Some(guard).

Now two CPUs have a &mut to COUNTER at once: no mutual exclusion, a race. E.g. both add 1 to COUNTER = 5: both read 5, both write 6 → one update is lost.

compare_exchange does "check it's false and set it to true" as one atomic step. Only one CPU can win false → true; the other gets Err and returns None.

(b) In 34k, `alloc_pid` handed out process ids from a `static mut NEXTPID` counter. If two CPUs create processes at the same moment, they could both read the same value and get the *same* pid. To prevent that, a student moves the counter into a `SpinLock` and rewrites `alloc_pid`, which `allocproc` calls for every new process:

```
static NEXTPID: SpinLock<usize> = SpinLock::new(1);

fn alloc_pid() -> usize {
    let mut next = NEXTPID.lock();   // lock the counter
    *next += 1;                      // reserve a pid by advancing the counter
    *NEXTPID.lock() - 1              // return the pid we reserved
}
```

What happens the first time `alloc_pid()` is called, and why? Fix `alloc_pid` so it returns 1, then 2, and so on. By contrast, a one-statement update like `*NEXTPID.lock() += 1;` works fine, even in a loop: why doesn't it have the same problem?

It hangs forever on the first call, so allocproc never returns and no process is ever created. next still holds the lock: a guard unlocks only when it is dropped, at the end of alloc_pid. The second NEXTPID.lock() spins waiting for a release that can only come from next, so it waits for itself: a self-deadlock.

Fix: use the guard you already hold; never lock it twice.

let pid = *next; *next += 1; pid (or: *next += 1; *next - 1)

In *NEXTPID.lock() += 1; the guard is a temporary: it is dropped at the end of that statement, so each statement locks and unlocks once.

(c) `lock` takes `&self`, a *shared* reference, yet `alloc_pid` writes `*next += 1` through the guard. The guard reaches the data like this:

```
impl<T> DerefMut for SpinLockGuard<'_, T> {     // lets you write through *guard
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.lock.data.get() }   // data.get() gives a *mut T
    }
}
```

Which field of `SpinLock` makes writing through a shared reference possible? Why is that safe here: what stops two CPUs from writing the data at the same time?

data: UnsafeCell<T>: its get() hands out a *mut T even from a shared reference ("interior mutability"). UnsafeCell itself checks nothing.

The lock makes it safe. A guard exists only after its CPU won the compare_exchange from false to true, and no other CPU can win until that guard is dropped and unlock() stores false. So at most one &mut to the data exists at a time: mutual exclusion.

2. **Semaphores and `Arc` (38k).** The 38k semaphore keeps its permit count in your spinlock:

```
pub struct Semaphore { count: SpinLock<i64> }

pub fn try_wait(&self) -> bool {       // take a permit if there is one
    let mut count = self.count.lock();
    if *count > 0 { *count -= 1; true } else { false }
}
pub fn post(&self) { *self.count.lock() += 1; }         // give one back
pub fn available(&self) -> i64 { *self.count.lock() }
```

Several parts of the kernel can share one semaphore through an `Arc` (shared ownership):

```
Arc::new(x)              // put x on the heap, with 1 owner
Arc::clone(&a)           // another owner of the SAME value (no copy): owners + 1
drop(a)                  // one fewer owner; the value is freed when owners reach 0
Arc::strong_count(&a)    // the current number of owners
```

(a) Fill in the table for this code. Leave "returns" blank for lines that return nothing. Then answer: what is unusual after line 6, and does any code stop it?

```
let sem = Arc::new(Semaphore::new(1));  // 1
let s2  = Arc::clone(&sem);             // 2
let a = sem.try_wait();                 // 3
let b = s2.try_wait();                  // 4
s2.post();                              // 5
s2.post();                              // 6
let c = sem.try_wait();                 // 7
drop(s2);                               // 8
let d = sem.try_wait();                 // 9
```

Line	returns	`available()`	owners (`strong_count`)
1		1	1
2		1	2
3	true	0	2
4	false	0	2
5		1	2
6		2	2
7	true	1	2
8		1	1
9	true	0	1

sem and s2 are two owners of one semaphore, so every try_wait and post changes the same count; only Arc::new, clone and drop change the owner count.

After line 6 there are 2 permits, though it started with 1: post just adds 1, and nothing caps the count. A semaphore meant to let one user in now lets two in.

(b) Does this compile?

```
let c  = Arc::new(5i64);
let c2 = Arc::clone(&c);
*c2 += 1;
```

Why can `s2.post()` change a count shared through an `Arc`, when `*c2 += 1` can't? Using the `SpinLock` below (from 37k), change the type of `c` so the increment works, and write the new increment.

```
// SpinLock<T> from 37k
impl<T> SpinLock<T> {
    pub const fn new(data: T) -> SpinLock<T>;     // a lock holding data
    pub fn lock(&self) -> SpinLockGuard<'_, T>;   // take the lock; returns a guard
}
// Read or write the value through the guard as *guard.
// The lock is released when the guard is dropped (end of statement or scope).
```

No: E0594: cannot assign to data in an `Arc`. An Arc only ever gives a shared &T (Deref, never DerefMut), because other owners may exist.

post takes &self too. It can change the count because the count sits in a SpinLock: its UnsafeCell gives interior mutability, and the lock makes that safe.

Fix: put the value in a lock, the same Arc<lock> pattern as Arc<Semaphore>:

```
let c  = Arc::new(SpinLock::new(5i64));
let c2 = Arc::clone(&c);
*c2.lock() += 1;        // c sees 6
```

3. **The kernel heap (38k).** 38k's `GlobalAlloc` serves every `Box`, `Vec` and `Arc` from your 32k page allocator:

```
unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
    if layout.size() > PGSIZE || layout.align() > PGSIZE {   // PGSIZE = 4096
        return ptr::null_mut();
    }
    kalloc::kalloc()                // one whole page per allocation
}
unsafe fn dealloc(&self, ptr: *mut u8, _layout: Layout) {
    kalloc::kfree(ptr);
}

struct Run { next: *mut Run }          // each free page starts with a link to the next
static mut FREELIST: *mut Run = ptr::null_mut();   // head of the free list
pub unsafe fn kfree(pa: *mut u8) {     // push page pa onto the front of the free list
    let r = pa as *mut Run;
    (*r).next = FREELIST;
    FREELIST = r;
}
pub unsafe fn kalloc() -> *mut u8 {    // pop the page at the head (null if empty)
    let r = FREELIST;
    if !r.is_null() { FREELIST = (*r).next; }
    r as *mut u8
}

Box::new(x), Vec::with_capacity(n)  // one alloc; dealloc when dropped
Arc::new(x)        // one alloc, holding x and its owner count (starts at 1)
Arc::clone(&a)     // another owner of the SAME value (no copy): owners + 1
drop(a)            // for an Arc: one fewer owner; dealloc when owners reach 0
```

Right after `kalloc::init()`, the free list starts `0x87fff000` → `0x87ffe000` → `0x87ffd000` → …

(a) Fill in the table. For each line, give the page it allocates or frees (write "none" if it does neither), and the page at the head of the free list afterwards.

```
let a  = Box::new(7u64);              // 1
let s  = Arc::new(Semaphore::new(2)); // 2
let s2 = Arc::clone(&s);              // 3
drop(a);                              // 4
let b  = Box::new(8u64);              // 5
drop(s);                              // 6
drop(s2);                             // 7
let v = Vec::<u64>::with_capacity(4); // 8
```

Line	allocates	frees	head after
1	0x87fff000	none	0x87ffe000
2	0x87ffe000	none	0x87ffd000
3	none	none	0x87ffd000
4	none	0x87fff000	0x87fff000
5	0x87fff000	none	0x87ffd000
6	none	none	0x87ffd000
7	none	0x87ffe000	0x87ffe000
8	0x87ffe000	none	0x87ffd000

The free list is last in, first out, so a freed page is the next one handed out (lines 4 → 5 and 7 → 8). Line 3 copies no data: it bumps the count. Line 6 only drops the count from 2 to 1; the page goes back when the last owner drops (line 7). (The Semaphore itself sits at 0x87ffe010: the Arc's two counts take the first 16 bytes of its page.)

(b) What happens, and why, in each case?

(i) `Vec::<u64>::with_capacity(512)`, and then `Vec::<u64>::with_capacity(513)`.

(ii) `kmain` forgets to call `kalloc::init()` before the first `Arc::new`.

(iii) How many bytes of its page does line 1's `Box` use?

(i) 512 × 8 = 4096 bytes = one page → works. 513 × 8 = 4104 > PGSIZE → alloc returns null, and Rust treats a failed allocation as fatal → the kernel panics.

(ii) The free list is still empty, so kalloc() returns null for the very first allocation → the same panic. The heap is built on top of the page allocator, so the page allocator has to be set up first.

(iii) A u64 is 8 bytes of the 4096: one page per allocation wastes the rest of the page.